

Generative AI Elevate your Software Development Career

Maulana Hafidz Ismail

25 Oktober 2025

Contents

1	Generative AI and Software Development	1
1.1	Beberapa Tokoh	1
1.2	Definisi Generative AI	1
1.3		1
1.4	Tren Masa Depan untuk AI dan Pengembangan Perangkat Lunak	2
1.5	Fase dalam SDLC	2
1.6	Penggunaan AI dalam SDLC	3
1.7	Penerapan AI di Setiap Fase	3
1.8	Definisi dan Konsep Utama LLM	3
1.9	Peran LLM dalam Pengembangan Perangkat Lunak	3
1.10	Kemampuan LLM	4
1.11	Pertimbangan Etis dan Tantangan	4
1.12	Tren Masa Depan untuk LLM	4
1.13	Pengertian Natural Language Processing (NLP)	4
1.14	Komponen dan Teknik NLP	4
1.15	Perpustakaan dan Alat NLP	5
1.16	Penggunaan NLP dalam Pengembangan Perangkat Lunak	5
1.17	Implikasi Etis dari NLP	6
1.18	Contoh Gen AI untuk Software Dev	6
1.19	Apa itu Token dalam API AI?	6
1.19.1	Token Tools	7
1.19.2	Bagaimana cara menghitung Token?	7
1.19.3	Petunjuk OpenAI dalam Menghitung Token	8
1.19.4	Contoh Perhitungan Token	8
1.20	Penggunaan Gen AI dalam Pengembangan Perangkat Lunak	9
1.21	Automasi dalam Testing	10
1.22	Contoh Alat Gen AI	10
1.23	memanfaatkan AI untuk Bantuan Teknis	10
1.24	Implementasi Design Patterns dengan Pendekatan AI	10
1.25	AI dan Arsitektur Perangkat Lunak	10
1.26	Tren Masa Depan dalam Pengembangan Perangkat Lunak	11
1.27	Pemanfaatan AI dalam Proses Desain Perangkat Lunak	11
1.28	Interaksi dengan Pengembang	11
1.29	Contoh Penggunaan AI	11
1.30	Alat dan Fitur AI	12
1.31	Penggunaan AI dalam Pengembangan Perangkat Lunak	12
1.32	Alat AI Populer untuk Pengembangan Perangkat Lunak	12
1.33	Proses AI dalam Pembuatan Situs Web Statis	13
1.34	Keuntungan dan Kerugian Situs Web Statis yang Dihasilkan AI	13
1.35	Alat AI Populer untuk Pembuatan Situs Web	13
1.36	Diagram Arsitektur dan Desain dengan Alat Kecerdasan Buatan Generatif	14
1.37	Pembuatan Diagram Arsitektur dan Desain: Kasus Penggunaan	15
1.37.1	Pembuatan Diagram Otomatis	15

1.37.2	Analisis Dependency	15
1.37.3	Pengenalan Pola	16
1.38	Menghasilkan Diagram dengan Menggunakan Generative AI . . .	16
1.39	Modernisasi Kode Lawas dengan Kecerdasan Buatan Generatif .	17
1.40	Peran Gen AI terhadap Kode Lawas	17
1.41	IBM watsonx Code Assistant untuk Z	18
2	Generative AI for Software Development Workflows and its Considerations	21
2.1	Definisi CI/CD	21
2.2	Penggunaan AI dalam CI/CD	21
2.3	Beberapa alat CI/CD yang didukung AI	21
2.4	Tren Masa Depan dalam CI/CD	22
2.5	Penggunaan Gen AI dalam Keamanan Perangkat Lunak	22
2.6	Alat dan Fungsionalitas AI dalam Keamanan	22
2.7	Praktik Terbaik dalam Integrasi Gen AI	22
2.8	Tantangan dan Pertimbangan Etis	22
2.9	Peran AI dalam Keamanan Perangkat Lunak	23
2.10	Aplikasi AI dalam Cybersecurity	23
2.11	Kelemahan AI dalam Cybersecurity	23
2.12	Integrasi AI dalam Pengembangan Perangkat Lunak	23
2.13	Alat Keamanan Kode	23
2.14	Istilah Teknis Security	24
2.15	Penerapan AI dalam Pengujian Perangkat Lunak	24
2.16	Natural Language Processing (NLP) dalam Pengujian	24
2.17	Contoh Kasus Penggunaan AI dalam Testing	25
2.18	Praktik Terbaik dalam Menggunakan AI untuk Pengujian	25
2.19	AI dalam Software Code Review	25
2.20	AI dalam Software Debugging	25
2.21	AI dalam Software Documentation	26
2.22	AI dalam Pendidikan dan Pelatihan	26
2.23	Manfaat dan tantangan penggunaan Kecerdasan Buatan Generatif dalam pengembangan perangkat lunak	26
2.24	Pertimbangan hak kekayaan intelektual	27
2.24.1	Strategi untuk melindungi hak kekayaan intelektual saat menggunakan kecerdasan buatan generatif	27
2.25	Pertimbangan keamanan	28
2.25.1	Mengurangi risiko kebocoran data dan akses tidak sah ke sistem kecerdasan buatan generatif	28
2.26	Pertimbangan kepatuhan	28
2.27	Mengintegrasikan transparansi dan akuntabilitas ke dalam sistem kecerdasan buatan generatif	29
2.28	Menangani ketidakakuratan dan bias	30
2.29	Pertimbangan Etis dalam Penggunaan Kecerdasan Buatan Generatif pada Pengembangan Perangkat Lunak	30

1 Generative AI and Software Development

1.1 Beberapa Tokoh

- Paul Pardi - Ahli materi di SkillUp Technology.
- Shubhra Das - Ahli IT di SkillUp Tech.
- Upkar Lidder - Manajer produk.
- Michael Rudnitski - Insinyur perangkat lunak di IBM.
- Ramesh Sannareddy - Direktur di Mongo Factory.

1.2 Definisi Generative AI

Generative AI adalah penggunaan teknik artificial intelligence untuk menghasilkan kode dan konten baru, seperti gambar, teks, atau musik yang menyerupai data yang ada.

1.3

- Automasi
Generative AI dapat digunakan untuk mengotomatiskan tugas-tugas berulang, mengurangi usaha manual, dan meningkatkan produktivitas. Contoh: Menggunakan AI untuk mengotomatiskan pengujian perangkat lunak.
- Code Optimization
AI dapat menganalisis basis kode yang besar untuk mengidentifikasi area yang dapat dioptimalkan guna meningkatkan kinerja atau mengurangi penggunaan memori. Contoh: Menggunakan algoritma AI untuk menyarankan perbaikan pada kode.
- Bug Detection
Dengan machine learning, model dapat dilatih untuk mengidentifikasi bug dan kerentanan umum dalam kode. Contoh: Menggunakan model untuk menganalisis pola dan laporan bug sebelumnya.
- Enhancing User Experience
Natural Language Processing (NLP) dapat digunakan untuk menganalisis umpan balik pengguna dan mengekstrak wawasan berharga. Contoh: Menggunakan NLP untuk memahami sentimen pengguna terhadap aplikasi.

- **Augmenting Creativity**
Generative models, seperti Generative Adversarial Networks (GANs), dapat digunakan untuk menciptakan desain baru dan inovatif. Contoh: Menggunakan GANs untuk menghasilkan antarmuka pengguna yang menarik.
- **Smart Documentation**
Sistem dokumentasi pintar dapat menganalisis kode dan komentar untuk menghasilkan dokumentasi yang deskriptif secara otomatis. Contoh: Menggunakan AI untuk membuat dokumentasi dari kode yang ada.

1.4 Tren Masa Depan untuk AI dan Pengembangan Perangkat Lunak

- **Efficiency Enhancement:** Generative AI dapat meningkatkan efisiensi dengan mengotomatiskan tugas yang memakan waktu.
- **Creative Collaboration:** Kolaborasi antara manusia dan model generatif dapat menghasilkan ide-ide baru.
- **Low-Code/No-Code Platforms:** Platform ini memungkinkan pengguna membangun aplikasi dengan sedikit pengetahuan coding.
- **Explainable AI:** AI yang dapat menjelaskan proses pengambilan keputusan model.
- **Intelligent Assistants:** Asisten cerdas yang lebih maju akan membantu pengembang selama siklus hidup pengembangan perangkat lunak.
- **Ethical AI Development:** Penekanan pada praktik pengembangan AI yang etis dan adil.

1.5 Fase dalam SDLC

- **Requirements Gathering and Analysis:** AI membantu mengotomatiskan analisis kebutuhan pengguna dan mengidentifikasi pola dalam dataset besar.
- **Design:** AI memberikan rekomendasi untuk arsitektur perangkat lunak yang optimal dan menghasilkan dokumentasi desain berdasarkan data historis.
- **Development:** AI mengotomatiskan tugas pengkodean yang berulang dan membantu dalam tinjauan kode untuk mengidentifikasi bug atau kerentanan.
- **Testing:** AI mengotomatiskan pembuatan dan eksekusi test case, serta menganalisis perubahan kode untuk mengidentifikasi area yang terpengaruh.

- **Deployment:** AI membantu mengotomatiskan proses deployment dan memantau kinerja perangkat lunak yang telah dideploy.
- **Maintenance:** AI menganalisis log dan umpan balik pengguna untuk mengidentifikasi pola dan memprediksi masalah potensial.

1.6 Penggunaan AI dalam SDLC

- **Natural Language Processing (NLP):** Teknologi yang memungkinkan komputer untuk memahami dan memproses bahasa manusia. Dalam konteks ini, NLP digunakan untuk mengekstrak informasi dari kebutuhan pengguna.
- **Machine Learning (ML):** Cabang dari AI yang memungkinkan sistem untuk belajar dari data dan meningkatkan kinerjanya seiring waktu. Dalam SDLC, ML digunakan untuk menganalisis data dan membuat prediksi.

1.7 Penerapan AI di Setiap Fase

- **Requirements Gathering:** NLP digunakan untuk mengekstrak informasi penting dari kebutuhan pengguna dan mengidentifikasi pola.
- **Design and Architecture:** AI menganalisis data untuk memberikan rekomendasi desain yang lebih baik dan mengidentifikasi pola desain yang sukses.
- **Code Generation:** AI dapat menghasilkan potongan kode berdasarkan basis kode yang ada, mempercepat proses pengkodean.
- **Testing and Quality Assurance:** AI mengotomatiskan pembuatan test case dan membantu dalam prediksi dan pencegahan bug.
- **DevOps:** Integrasi AI dalam DevOps memungkinkan pengambilan keputusan otomatis berdasarkan data real-time, meningkatkan efisiensi dan keandalan.

1.8 Definisi dan Konsep Utama LLM

Large Language Models (LLMs) adalah model AI yang dilatih untuk memahami dan menghasilkan teks yang koheren dan relevan secara kontekstual. Mereka menggunakan teknik deep learning pada dataset besar yang berisi teks dari berbagai sumber.

1.9 Peran LLM dalam Pengembangan Perangkat Lunak

- **Code Generation and Auto-completion:** LLMs membantu dalam menghasilkan kode dan menyarankan penyelesaian kode, mengurangi usaha manual dan mempercepat proses pengkodean.

- Automated Bug Detection and Fixing: LLMs dapat mengidentifikasi bug dalam kode dengan menganalisis sintaks dan semantik, serta memberikan saran perbaikan.

1.10 Kemampuan LLM

- Generating Code Templates and Snippets: LLMs dapat membuat template dan potongan kode berdasarkan kebutuhan spesifik, membantu pengembang menghemat waktu.
- Assisting with Documentation and Comments: LLMs menganalisis fungsi kode untuk menghasilkan dokumentasi dan komentar, meningkatkan keterbacaan kode.

1.11 Pertimbangan Etis dan Tantangan

- Bias Detection: LLMs dapat mewarisi bias dari data pelatihan mereka, sehingga penting untuk mendeteksi dan mengurangi bias ini untuk mencegah hasil yang tidak adil.
- Security and Privacy: Penggunaan LLM harus mempertimbangkan keamanan dan privasi kode untuk melindungi dari akses yang tidak sah.

1.12 Tren Masa Depan untuk LLM

- Integration with IDEs: Penelitian sedang dilakukan untuk mengintegrasikan LLM dengan Integrated Development Environments (IDEs) untuk memberikan akses mudah ke fitur yang didukung LLM.
- Collaboration with Developers: Kerjasama antara pengembang dan LLM akan terus berkembang, dengan pengembang membimbing keluaran LLM selama berbagai tahap pengembangan perangkat lunak.

1.13 Pengertian Natural Language Processing (NLP)

Natural Language Processing adalah teknik komputasi yang digunakan untuk menganalisis, memahami, dan menghasilkan bahasa manusia. Ini menggabungkan linguistik, ilmu komputer, dan kecerdasan buatan untuk memproses data bahasa alami.

1.14 Komponen dan Teknik NLP

- Linguistics Concepts: Dasar dari NLP yang mencakup:
 - Morphology: Studi tentang struktur kata.
 - Syntax: Studi tentang struktur kalimat.
 - Semantics: Studi tentang makna.

- Text Processing and Cleaning Techniques: Langkah penting dalam NLP yang meliputi:
 - Tokenization: Proses membagi teks menjadi unit-unit yang lebih kecil yang disebut tokens (kata, kalimat, atau karakter).
 - Stemming: Teknik yang mengurangi kata ke bentuk dasarnya untuk menyederhanakan kosakata.

1.15 Perpustakaan dan Alat NLP

Beberapa perpustakaan dan alat NLP yang umum digunakan:

- IBM Watson
- Aylien
- Google Cloud NLP API
- NLTK: Perpustakaan Python yang paling populer untuk NLP.
- MonkeyLearn
- Amazon Comprehend
- Stanford Core NLP
- TextBlob
- SpaCy
- GenSim

1.16 Penggunaan NLP dalam Pengembangan Perangkat Lunak

- Sentiment Analysis: Proses menentukan emosi yang diekspresikan dalam teks dengan memberikan skor polaritas pada kata atau frasa.
- Named Entity Recognition (NER): Mengidentifikasi dan mengklasifikasi entitas bernama dalam teks ke dalam kategori yang telah ditentukan.
- Text Classification: Tugas mengkategorikan teks ke dalam label atau kategori yang telah ditentukan.
- Machine Translation: Teknik yang memungkinkan komputer untuk memahami dan menghasilkan bahasa manusia.
- Chatbots: Program komputer yang mensimulasikan percakapan manusia dengan menggunakan teknik NLP untuk memahami pertanyaan pengguna dan menghasilkan respons yang sesuai.

1.17 Implikasi Etis dari NLP

NLP juga menimbulkan kekhawatiran etis terkait privasi, bias, keadilan, transparansi, dan akuntabilitas. Penting untuk memastikan keadilan dalam algoritma untuk mencegah diskriminasi dan menggunakan NLP secara bertanggung jawab dengan mempertimbangkan pertimbangan etis sepanjang siklus pengembangan.

1.18 Contoh Gen AI untuk Software Dev

- ChatGPT adalah karya OpenAI yang dikenal karena kemampuannya menghasilkan teks yang mirip manusia. ChatGPT unggul dalam percakapan alami, penjelasan pertanyaan, dan dukungan penulisan kreatif.
- Watsonx.ai membantu memproses volume besar data teks, menghemat waktu dan sumber daya, serta kemampuannya dalam penerjemahan bahasa memfasilitasi komunikasi dan kolaborasi global.
- GPT-4 adalah model bahasa AI canggih yang membantu menghasilkan teks berkualitas tinggi, meningkatkan penciptaan konten, dan memperkuat tugas pemrosesan bahasa alami.
- Bard adalah chatbot dan alat penciptaan konten terdepan yang dikembangkan oleh Google. Ia memanfaatkan LaMDA, model berbasis transformer, untuk membantu pengembangan perangkat lunak, pertanyaan pemrograman, penulisan konten, penelitian, dan belajar.

1.19 Apa itu Token dalam API AI?

Token adalah bagian penting dari API ChatGPT. Token adalah potongan atau segmen kata. Sebelum memproses prompt, API memecah masukan menjadi token-token individu ini.

Token-token ini tidak selalu sejalan dengan awal atau akhir kata — mereka dapat mencakup spasi di akhir dan bahkan sub-kata.

Token Limits: Biasanya, requests dapat menggunakan hingga 4097 token yang dibagi antara prompt dan completion, tergantung pada modelnya. Misalnya, jika prompt Anda terdiri dari 4000 token, completion Anda dapat berisi maksimal 97 token. Batasan ini merupakan kendala teknis, tetapi seringkali ada cara kreatif untuk mengatasinya, seperti mengompres prompt Anda atau membagi teks menjadi potongan-potongan yang lebih kecil.

Token Pricing: API menyediakan berbagai jenis model dengan harga yang berbeda-beda. Setiap model memiliki rentang kemampuan yang berbeda, dengan "gpt-3.5-turbo" sebagai yang paling canggih. Permintaan yang ditujukan ke model-model ini memiliki harga yang berbeda. Informasi detail mengenai harga token tersedia di halaman API produk.

Exploring Tokens: API memproses kata berdasarkan konteksnya dalam data korpus. GPT-3 mengambil prompt, mengubah input menjadi daftar token, memproses prompt, dan kemudian mengubah token yang diprediksi kembali

menjadi kata-kata sebagai respons. Perlu dicatat bahwa dua kata yang tampak identik dapat dihasilkan sebagai token yang berbeda tergantung pada strukturnya dalam teks. Misalnya, pertimbangkan bagaimana API menghasilkan nilai token untuk kata "red" berdasarkan konteksnya.

1.19.1 Token Tools

Alat yang paling populer di antara semua alat adalah alat tokenizer interaktif OpenAI. Alat ini membantu menghitung jumlah token dan mengamati bagaimana teks dibagi menjadi token.

Jika perlu melakukan tokenisasi teks secara programatik, gunakan Tiktoken, sebuah tokenizer BPE yang cepat dan dirancang khusus untuk model OpenAI.

Perpustakaan lain termasuk paket transformers untuk Python atau paket gpt-3-encoder untuk Node.js.

1.19.2 Bagaimana cara menghitung Token?

Untuk menghitung token saat memanggil OpenAI, ikuti langkah-langkah berikut:

- Identifikasi endpoint API: Tentukan endpoint OpenAI mana yang akan Anda panggil.
- Setiap endpoint mewakili fungsi atau sumber daya spesifik yang disediakan oleh API.
- Periksa dokumentasi API: Akses dokumentasi API yang disediakan oleh penyedia API. Dokumentasi tersebut akan menjelaskan struktur permintaan dan respons API, termasuk header, parameter, atau metode autentikasi yang diperlukan.
- Periksa apakah API memerlukan otentikasi berbasis token: Otentikasi berbasis token melibatkan penambahan token akses ke header permintaan untuk mengotorisasi panggilan API. Token ini biasanya diperoleh dengan mendaftarkan aplikasi ke penyedia API dan mengikuti proses otentikasi mereka.
- Peroleh token akses: Jika otentikasi berbasis token diperlukan, peroleh token akses dengan mengikuti proses otentikasi yang ditentukan oleh penyedia API. Proses ini mungkin melibatkan pendaftaran aplikasi, penyediaan kredensial, dan penerimaan token.
- Hitung token untuk setiap API: Setelah Anda memiliki token akses, hitunglah sebagai satu token untuk setiap panggilan API yang Anda lakukan. Setiap permintaan ke titik akhir API, termasuk token akses dalam header, dihitung sebagai satu token.

- Pantau penggunaan token: Pantau jumlah token yang telah Anda gunakan untuk memastikan Anda tetap dalam batas penggunaan atau kuota yang ditetapkan oleh penyedia API. Beberapa API mungkin membatasi jumlah token yang dapat Anda gunakan dalam periode tertentu.

1.19.3 Petunjuk OpenAI dalam Menghitung Token

Berdasarkan dokumen resmi OpenAI, berikut adalah perkiraan jumlah token:

- 1 token = 4 karakter dalam bahasa Inggris
- 1 token = $\frac{3}{4}$ kata
- 100 token = 75 kata
- 1-2 kalimat = 30 token
- 1 paragraf = 100 token
- 1.500 kata = 2.048 token

1.19.4 Contoh Perhitungan Token

Misalnya, Anda memiliki endpoint OpenAI yang memerlukan permintaan GET untuk mengambil informasi pengguna. Endpoint tersebut memiliki detail sebagai berikut:

- Metode Permintaan: GET
- URL Endpoint: `https://api.example.com/users/userId`
- Parameter Jalan: `userId` (nilai: "123")
- Parameter Query: `includeAddress` (nilai: true)

Menghitung token untuk permintaan ini:

- Metode permintaan 'GET' biasanya memerlukan 1 token.
- URL endpoint tidak mengonsumsi token tambahan.
- Parameter jalur 'userId' dengan nilai '123' tidak mengonsumsi token tambahan.
- Parameter kueri 'includeAddress' dengan nilai 'true' mengonsumsi 2 token (1 token untuk nama parameter dan 1 token untuk nilai parameter).
- Jumlah total token dari langkah 1 hingga 4, jumlah token total untuk permintaan API ini adalah 3 token.

Catatan: Metode penghitungan token yang tepat dapat bervariasi tergantung pada implementasi API spesifik atau penyedia API. Konsultasikan dokumentasi API untuk informasi akurat tentang penggunaan token dan biaya atau batasan yang terkait.

Contoh: Menghitung biaya untuk prompt masukan dan keluaran menggunakan rumus perhitungan biaya sampel.

Prompt masukan: Prompt: “Bisakah Anda memberikan gambaran singkat tentang manfaat berolahraga?”

- Langkah 1:

Tentukan jumlah kata dalam prompt.

Dalam hal ini, prompt memiliki 9 kata.

- Langkah 2:

Hitung biaya untuk prompt berdasarkan jumlah token. Asumsikan biaya per 1000 token adalah \$0,03.

Karena prompt memiliki 9 kata dan tingkat konversi rata-rata adalah 750 kata per 1000 token, biaya prompt = $(9 / 750) * (0,03) = \$0,00036$

- Langkah 3:

Jika model AI menghasilkan output sekitar 500 kata, gunakan rumus yang sama untuk menghitung biaya menghasilkan output. Biaya output = $(500 / 750) * (0,06) = \$0,04$

- Langkah 4:

Hitung biaya total. Biaya total diperkirakan = Biaya prompt + Biaya output = $\$0,00036 + \$0,04 = \$0,04036$

Oleh karena itu, biaya diperkirakan untuk menghasilkan output sekitar 500 kata menggunakan prompt yang diberikan adalah \$0,04036.

Demikian pula, jika aplikasi memanggil API 1000 kali sehari, biayanya ditampilkan sebagai berikut:

Biaya per hari = \$40,36 Biaya per bulan = \$1.210,80

Catatan: Biaya aktual dapat bervariasi tergantung pada jumlah token yang digunakan dalam menghasilkan konten dan jenis model yang digunakan.

1.20 Penggunaan Gen AI dalam Pengembangan Perangkat Lunak

- Code Generation: Generative AI dapat secara otomatis menulis potongan kode berdasarkan kebutuhan input, mengurangi waktu yang dihabiskan pengembang untuk tugas pengkodean yang berulang.
- Bug Detection: AI dapat memindai kode untuk mendeteksi masalah potensial dan menawarkan solusi, membantu pengembang menemukan dan memperbaiki bug lebih cepat.

1.21 Automasi dalam Testing

- Test Case Generation: AI dapat mengotomatiskan pembuatan test case, yang membantu dalam mengidentifikasi dan memperbaiki masalah dengan lebih cepat.
- Automated Testing Tools: Alat seperti Testim dapat menghasilkan dan mengeksekusi test case berdasarkan alur pengguna.

1.22 Contoh Alat Gen AI

- GitHub Copilot: Alat ini membantu pengembang, terutama yang junior, untuk menulis metode, kelas, dan fungsi lebih cepat dengan memberikan saran potongan kode berdasarkan niat fungsi.
- ChatGPT: Dapat digunakan untuk berbagai tugas generative AI, termasuk menghasilkan dokumentasi dari kode.

1.23 memanfaatkan AI untuk Bantuan Teknis

- AI-Powered Code Completion Tools: Alat ini membantu dalam menyelesaikan kode dengan memberikan saran yang akurat, menghemat waktu dan meningkatkan akurasi pengkodean.
- Automated Code Review and Bug Detection: AI dapat menilai kualitas kode, menemukan bug potensial, dan merekomendasikan perbaikan.

1.24 Implementasi Design Patterns dengan Pendekatan AI

- Singleton Design Pattern: Digunakan untuk memastikan hanya ada satu instance dari kelas tertentu, seperti koneksi database.

```
class DatabaseConnection { private static instance;  
    public static getInstance() { ... } }
```

- Observer Pattern: Memungkinkan objek untuk memberi tahu objek lain tentang perubahan status, seperti dalam aplikasi chat. Definisi: Pola desain yang memungkinkan satu objek mengawasi dan merespons perubahan pada objek lain.

1.25 AI dan Arsitektur Perangkat Lunak

- Generating High-Level Architecture from Code: AI dapat menghasilkan arsitektur tingkat tinggi dari kode yang ada, memberikan wawasan tentang struktur sistem. Definisi: Proses menciptakan gambaran umum dari arsitektur perangkat lunak berdasarkan kode yang ada.

- Real-Time Architecture Updates: Memungkinkan pembaruan arsitektur secara langsung untuk meningkatkan kolaborasi dalam proyek pengembangan perangkat lunak. Definisi: Pembaruan yang dilakukan secara langsung untuk mencerminkan perubahan dalam arsitektur.

1.26 Tren Masa Depan dalam Pengembangan Perangkat Lunak

- AI-Driven Tools and Platforms: Alat yang didorong oleh AI akan mendukung pengembang dengan mengotomatiskan tugas dan menghasilkan prototipe.
- Quantum Computing Integration: Integrasi komputasi kuantum dapat merevolusi pengembangan perangkat lunak dengan menyelesaikan masalah kompleks. Penggunaan komputer kuantum untuk memecahkan masalah yang tidak dapat diselesaikan oleh komputer klasik.

1.27 Pemanfaatan AI dalam Proses Desain Perangkat Lunak

- Automasi Desain: AI digunakan untuk menganalisis kebutuhan dan menghasilkan pola desain yang sesuai dengan praktik terbaik, sehingga membuat fase awal desain lebih efisien.
- Pengurangan Kesalahan Manual: Dengan menggunakan AI, tim dapat mengurangi kesalahan manual, meningkatkan konsistensi desain, dan mempercepat proses pengembangan.

1.28 Interaksi dengan Pengembang

- Dialog dengan Developer: Gen AI dapat berkomunikasi dengan pengembang, memungkinkan mereka untuk memberikan deskripsi tentang apa yang ingin dicapai dalam solusi perangkat lunak.
- Pembuatan Alur Logika: AI dapat membuat deskripsi alur logika yang menunjukkan bagaimana data akan bergerak melalui aplikasi dan operasi yang perlu dibuat oleh pengembang.

1.29 Contoh Penggunaan AI

- Analisis Proyek Serupa: Alat seperti OpenAI Codex dapat menghasilkan kode contoh untuk arsitektur tipikal, seperti MVP atau MVVP.
- Prototipe Cepat: Gen AI digunakan untuk membuat desain UI/UX dan prototipe cepat yang dapat ditunjukkan kepada klien.

1.30 Alat dan Fitur AI

- Refact.ai: Alat ini membantu menyusun kode dengan mendeteksi dan merombak blok kode yang berulang, meningkatkan keterbacaan dan kinerja.
- DiagramGPT dan LeonardoAI: Alat ini dapat mengubah deskripsi bahasa alami menjadi diagram arsitektur yang dapat dibangun dan ditingkatkan oleh pengembang.
- Lucidchart: Alat untuk membuat diagram yang dapat mengintegrasikan fitur AI untuk membantu menghasilkan diagram sistem atau jaringan secara otomatis berdasarkan data yang dimasukkan.
- Microsoft Visio: Alat diagram yang juga dapat memanfaatkan fitur AI untuk membantu dalam pembuatan diagram arsitektur dan visualisasi sistem.
- PlantUML: Alat yang memungkinkan pengembang untuk membuat diagram dari kode, yang dapat diintegrasikan dengan pipeline CI/CD untuk memperbarui diagram secara otomatis saat kode berubah.

1.31 Penggunaan AI dalam Pengembangan Perangkat Lunak

- Code Generation: AI dapat secara otomatis menghasilkan potongan kode berdasarkan kebutuhan yang diberikan, mengurangi waktu yang dihabiskan pengembang untuk tugas pengkodean yang berulang.
- Bug Detection: AI dapat memindai kode untuk mendeteksi potensi masalah dan menawarkan solusi, membantu pengembang menemukan dan memperbaiki bug lebih cepat.
- Technical Support: AI meningkatkan efisiensi dukungan teknis dengan menggunakan Natural Language Processing (NLP) untuk memahami dan merespons pertanyaan pengembang.

1.32 Alat AI Populer untuk Pengembangan Perangkat Lunak

- ChatGPT: Alat AI yang digunakan untuk menjawab pertanyaan dan membantu dalam penulisan serta debugging kode.
- GitHub Copilot: Alat AI yang berfungsi sebagai rekan pemrograman, memberikan saran kode secara real-time.
- Tabnine: Alat penyelesaian kode AI yang memberikan saran berdasarkan pola pengkodean yang dipelajari.

1.33 Proses AI dalam Pembuatan Situs Web Statis

- Pengumpulan Preferensi: Pengembang memberikan preferensi kepada AI melalui teks atau visual.
- Interpretasi Input: AI memahami skema warna, tata letak, tipografi, dan fitur yang diinginkan.
- Pembuatan Desain: AI menghasilkan desain situs dengan menghasilkan kode HTML dan CSS berdasarkan pilihan pengguna.
- Optimasi Kode: AI menyederhanakan kode untuk meningkatkan kinerja dan waktu muat situs.
- Ekspor Kode: Pengembang mengeksport kode yang dihasilkan dan mengunggahnya ke platform hosting web.

1.34 Keuntungan dan Kerugian Situs Web Statis yang Dihasilkan AI

- Keuntungan:
 - Penghematan Waktu: Mengurangi waktu pengembangan dengan menghilangkan pengkodean manual.
 - Standar Desain yang Konsisten: AI memastikan keseragaman desain di seluruh segmen situs.
 - Peningkatan Produktivitas: AI menangani tugas berulang, memungkinkan pengembang untuk fokus pada aspek lain.
 - Peningkatan Aksesibilitas: Situs yang dihasilkan AI mematuhi standar aksesibilitas.
 - Opsi Kustomisasi: Pengembang dapat menyesuaikan kode dan tata letak meskipun menggunakan AI.
- Kerugian:
 - Masalah Standarisasi: Tata letak dari pembuat situs web AI sering kali terlihat serupa.
 - Kontrol Terbatas: Beberapa pembuat AI mungkin menghasilkan situs yang terlihat generik dengan opsi kustomisasi yang minimal.

1.35 Alat AI Populer untuk Pembuatan Situs Web

- GPT (Generative Pre-trained Transformer 3): Model bahasa yang dapat menghasilkan kode HTML dan CSS.
- TeleportHQ: Platform front-end kolaboratif untuk pengembangan UI.

- Visily: Alat wireframe yang mengubah tangkapan layar menjadi prototipe yang dapat diedit.
- Framer X: Alat desain yang menerjemahkan desain visual menjadi kode siap produksi.
- Wix ADI: Pembuat situs web yang secara otomatis merancang situs berdasarkan preferensi pengguna.
- Webflow: Platform pengembangan web visual yang menggabungkan teknologi AI dengan antarmuka drag-and-drop.

1.36 Diagram Arsitektur dan Desain dengan Alat Kecerdasan Buatan Generatif

Diagram arsitektur perangkat lunak membantu menggambarkan struktur dan desain sistem perangkat lunak yang kompleks secara visual. Secara tradisional, pembuatan diagram ini memerlukan upaya manual yang signifikan dan keahlian khusus. Namun, dengan kemunculan alat dan algoritma kecerdasan buatan (AI), pembuatan diagram arsitektur perangkat lunak menjadi lebih efisien dan otomatis.

Berikut adalah beberapa alat dan teknik berbasis AI yang memudahkan pembuatan diagram arsitektur perangkat lunak.

- Pembuatan diagram berbasis Pemrosesan Bahasa Alami (NLP): Alat berbasis kecerdasan buatan (AI) yang dilengkapi dengan kemampuan pemrosesan bahasa alami dapat menganalisis deskripsi teks sistem perangkat lunak dan secara otomatis menghasilkan diagram arsitektur yang sesuai. Misalnya, dengan memasukkan deskripsi teks seperti “Sistem terdiri dari server web, server aplikasi, dan server basis data,” alat berbasis AI dapat menghasilkan diagram yang menggambarkan hubungan antar komponen tersebut.
- Analisis kode untuk pembuatan diagram: Algoritma kecerdasan buatan (AI) dapat menganalisis basis kode, mengidentifikasi modul-modul utama, komponen, dan hubungan di antara mereka, serta menghasilkan representasi visual dari arsitektur perangkat lunak. Pendekatan ini menghemat waktu dan memastikan akurasi dalam menangkap struktur sistem.
- Pengenalan gambar untuk reverse engineering: Algoritma kecerdasan buatan (AI) yang dilatih dalam pengenalan gambar membantu menghasilkan diagram arsitektur perangkat lunak dari representasi visual yang sudah ada. Misalnya, jika sistem warisan tidak memiliki dokumentasi yang lengkap, alat yang didukung AI dapat menganalisis tangkapan layar atau artefak visual lainnya, mengidentifikasi pola, dan menghasilkan diagram arsitektur yang mencerminkan struktur dasar sistem tersebut.

- Pembelajaran mesin untuk identifikasi pola: Teknik pembelajaran mesin menganalisis sejumlah besar diagram arsitektur perangkat lunak yang ada dan mengidentifikasi pola berulang atau prinsip desain. Dengan melatih model AI pada dataset tersebut, model tersebut dapat belajar mengenali pola arsitektur yang umum dan menghasilkan diagram baru berdasarkan pengetahuan yang diperoleh. Teknik ini mempercepat proses pembangkitan diagram dan membantu menjaga konsistensi di berbagai proyek.
- Pembuatan diagram kolaboratif dengan kecerdasan buatan (AI): Alat-alat yang didukung AI memfasilitasi pembuatan diagram kolaboratif dengan memungkinkan berbagai pemangku kepentingan untuk berkontribusi dengan wawasan dan pengetahuan mereka. Alat-alat ini membantu mengintegrasikan masukan dari berbagai anggota tim, menganalisisnya secara kolektif, dan menghasilkan diagram arsitektur perangkat lunak yang komprehensif yang menggabungkan berbagai perspektif. Pendekatan kolaboratif ini meningkatkan komunikasi dan keselarasan di antara anggota tim.

1.37 Pembuatan Diagram Arsitektur dan Desain: Kasus Penggunaan

1.37.1 Pembuatan Diagram Otomatis

Alat berbasis AI dapat secara otomatis menghasilkan diagram arsitektur perangkat lunak dengan menggunakan prompt, menganalisis basis kode yang mendasarinya, dan mengekstrak informasi yang relevan. Alat-alat ini menggunakan algoritma pembelajaran mesin untuk memahami hubungan antara komponen, modul, dan ketergantungan dalam kode. Mereka menciptakan diagram arsitektur tingkat tinggi yang memberikan gambaran umum tentang sistem perangkat lunak dengan menganalisis pola kode, konvensi penamaan, dan struktur kode.

Contoh: ChartAI, ChatUML, Eraser

Alat-alat berbasis AI ini menggunakan teknik pemrosesan bahasa alami (NLP) canggih untuk menghasilkan diagram alur dan diagram dari prompt. Mereka membantu pengembang memvisualisasikan arsitektur perangkat lunak dan mengidentifikasi pilihan desain potensial dengan mengubah prompt tersebut menjadi diagram alur visual.

1.37.2 Analisis Dependency

Memahami ketergantungan antara berbagai komponen sangat penting dalam merancang sistem perangkat lunak yang skalabel dan mudah dipelihara. Alat yang didukung AI dapat menganalisis basis kode untuk mengidentifikasi ketergantungan dan menghasilkan grafik atau diagram ketergantungan yang detail. Diagram-digram ini membantu pengembang memvisualisasikan interaksi antara berbagai modul dan komponen dalam arsitektur perangkat lunak.

Contoh: JArchitect adalah alat berbasis AI yang dirancang khusus untuk aplikasi Java. Alat ini melakukan analisis kode statis untuk mengidentifikasi keter-

gantungan antara kelas, paket, dan perpustakaan dalam proyek Java. Dengan memvisualisasikan ketergantungan ini dalam format grafik interaktif, JArchitect memungkinkan pengembang untuk memahami dampak perubahan pada arsitektur keseluruhan dan membuat keputusan yang terinformasi selama refactoring atau evolusi sistem.

1.37.3 Pengenalan Pola

Arsitektur perangkat lunak sering kali mengintegrasikan pola desain untuk memecahkan masalah dan meningkatkan kualitas sistem. Mengidentifikasi pola-pola ini secara manual dapat memakan waktu dan rentan terhadap kesalahan. Namun, alat yang didukung kecerdasan buatan (AI) dapat memanfaatkan algoritma pembelajaran mesin untuk mengenali dan mengekstrak pola desain dari basis kode secara otomatis.

Contoh: ArchR adalah alat berbasis AI yang menggunakan teknik pengenalan pola untuk mengidentifikasi pola arsitektur dalam sistem perangkat lunak. Dengan menganalisis basis kode, ArchR dapat mendeteksi pola umum seperti arsitektur berlapis, arsitektur klien-server, atau pola MVC (Model-View-Controller). Kemudian, ArchR menghasilkan diagram visual yang menyoroti pola-pola tersebut, sehingga memudahkan pengembang untuk memahami struktur keseluruhan sistem.

1.38 Menghasilkan Diagram dengan Menggunakan Generative AI

- ChartAI

ChartAI adalah aplikasi diagram berbasis AI yang membantu profesional membuat diagram berkualitas tinggi seperti sequence diagrams, Gantt charts, dan flowcharts.

1. Fungsi: Mengoptimalkan tugas perencanaan dengan teknologi AI untuk menyesuaikan timeline, mengidentifikasi ketergantungan, dan menghasilkan layout secara otomatis.
2. Penggunaan: Pengguna hanya perlu memberikan prompt untuk membuat diagram, misalnya, untuk sistem login pengguna.

- ChatUML

ChatUML adalah generator diagram yang dibantu AI yang menyederhanakan pembuatan diagram kompleks.

1. Fungsi: Menghasilkan diagram yang jelas dan menarik secara visual dengan cepat, serta kompatibel dengan PlantUML dan mermaid.js.
2. Contoh Penggunaan: Menggunakan prompt untuk menghasilkan diagram untuk sistem pendaftaran pengguna di situs e-commerce.

- Eraser

Eraser adalah platform dokumen dan diagram untuk tim teknik.

1. Fungsi: Memungkinkan pengembang membuat dokumen dan diagram dengan cepat melalui UI minimalis dan alur berbasis keyboard.
2. Contoh Penggunaan: Menggunakan contoh bawaan untuk membuat diagram arsitektur Cloud dan sequence diagram.

- ChatGPT

ChatGPT dapat digunakan untuk menghasilkan diagram dengan teks yang kompatibel dengan mermaid.js. Contoh Penggunaan: Menggambar sistem penyimpanan file berbasis cloud dan meminta ChatGPT untuk menghasilkan gambar.

1.39 Modernisasi Kode Lawas dengan Kecerdasan Buatan Generatif

Kode warisan merujuk pada basis kode yang telah dikembangkan seiring waktu dan mungkin sudah usang atau sulit untuk dipelihara. Namun, berkat kemajuan teknologi, pengelolaan dan modernisasi kode warisan menjadi lebih efisien dengan munculnya teknik Kecerdasan Buatan Generatif (AI).

Kode warisan dapat diidentifikasi melalui penggunaan bahasa pemrograman yang usang, kurangnya dokumentasi, arsitektur perangkat lunak yang buruk, dan ketergantungan pada teknologi yang sudah tidak digunakan lagi. Tantangan yang terkait dengan kode warisan meliputi skalabilitas yang terbatas, kemudahan pemeliharaan yang berkurang, dan risiko yang lebih tinggi terhadap bug dan kerentanan keamanan.

1.40 Peran Gen AI terhadap Kode Lawas

- Refactoring kode: Model kecerdasan buatan generatif dapat menganalisis struktur dan pola dalam kode warisan untuk mengusulkan perbaikan refactoring. Mereka secara otomatis mengidentifikasi blok kode yang berlebihan, mengoptimalkan algoritma, atau mengusulkan gaya penulisan kode alternatif yang sesuai dengan praktik terbaik modern. Hal ini membantu meningkatkan keterbacaan kode, mengurangi utang teknis, dan meningkatkan kualitas perangkat lunak secara keseluruhan. Misalnya, alat seperti DeepCode dapat mengidentifikasi potensi bug dan kerentanan dalam kode, memungkinkan pengembang untuk memperbaikinya dengan lebih efisien.
- Pembuatan dokumentasi: Model kecerdasan buatan generatif dapat menganalisis basis kode dan secara otomatis menghasilkan dokumentasi, termasuk deskripsi fungsi, referensi API, dan contoh penggunaan. Hal ini menghemat waktu dan tenaga pengembang dalam mendokumentasikan

basis kode secara manual, sehingga memudahkan anggota tim baru untuk memahami dan bekerja dengan kode yang sudah ada. Misalnya, alat seperti NaturalDocs dapat menghasilkan dokumentasi komprehensif dari komentar, variabel, dan tanda tangan fungsi, sehingga memudahkan pengembang untuk memahami dan bekerja dengan kode warisan.

- Deteksi dan penyelesaian bug: Kode warisan seringkali mengandung bug tersembunyi yang mungkin tidak terdeteksi dalam waktu lama. Model AI generatif dapat menganalisis pola kode dan mengidentifikasi area yang berpotensi rentan terhadap bug. Dengan mengidentifikasi masalah potensial sejak dini, pengembang dapat menanganinya secara proaktif sebelum berkembang menjadi masalah kritis. Selain itu, AI generatif juga dapat menyarankan perbaikan atau tambalan untuk bug yang teridentifikasi berdasarkan pola yang diamati dalam skenario serupa. Misalnya, alat seperti "DeepCode" dapat menghasilkan potongan kode atau fungsi lengkap berdasarkan konteks kode warisan, menghemat waktu dan usaha pengembang.
- Migrasi ke teknologi modern: Sistem warisan sering kali bergantung pada teknologi usang yang tidak lagi didukung atau efisien. AI generatif dapat membantu dalam memigrasikan kode warisan ke platform atau bahasa modern dengan secara otomatis menerjemahkan basis kode ke teknologi target yang diinginkan. Hal ini secara signifikan mengurangi upaya manual yang diperlukan untuk menulis ulang seluruh basis kode dari awal. Misalnya, alat seperti "Kite" menggunakan algoritma pembelajaran mesin untuk menyarankan potongan kode yang relevan, panggilan fungsi, dan nama variabel, sehingga meningkatkan produktivitas pengembang saat bekerja dengan kode warisan.

1.41 IBM watsonx Code Assistant untuk Z

Lebih dari 80% transaksi tatap muka di lembaga keuangan AS menggunakan COBOL. IBM telah mengidentifikasi 240 miliar baris kode COBOL yang berjalan saat ini, dengan tambahan 5 miliar baris ditulis setiap tahun.

Menurut penelitian dari IBM Institute for Business Value, organisasi lebih cenderung memanfaatkan aset mainframe yang ada daripada membangun ulang ekosistem aplikasi mereka dari awal dalam dua tahun ke depan. Namun, kekurangan sumber daya dan keterampilan tetap menjadi tantangan utama bagi organisasi-organisasi ini.

Terdapat berbagai pendekatan modernisasi aplikasi yang tersedia saat ini. Beberapa opsi termasuk menulis ulang seluruh kode aplikasi dalam Java atau memigrasikan semuanya ke cloud publik. Namun, pendekatan-pendekatan ini mungkin mengorbankan kemampuan inti IBM Z dan gagal memberikan pengurangan biaya yang diharapkan.

Alat yang mengubah aplikasi COBOL menjadi sintaks Java dapat menghasilkan kode yang sulit dipelihara dan tidak familiar bagi pengembang Java. Meskipun kecerdasan buatan generatif menunjukkan potensi, teknologi

penulisan ulang parsial yang didukung AI saat ini tidak mendukung COBOL dan tidak mengoptimalkan kode Java hasilnya untuk tugas-tugas spesifik.

IBM telah meluncurkan watsonx Code Assistant for Z, produk baru yang memanfaatkan kecerdasan buatan generatif untuk memfasilitasi penerjemahan COBOL ke Java dan meningkatkan produktivitas pengembang di platform tersebut.

Watsonx Code Assistant for Z merupakan penambahan baru dalam keluarga produk watsonx code assistant. Solusi-solusi ini akan didukung oleh model kode watsonx.ai IBM, yang memiliki pengetahuan tentang 115 bahasa pemrograman dan telah belajar dari 1,5 triliun token. Dengan 20 miliar parameter, model ini berpotensi menjadi salah satu model AI generatif terbesar untuk otomatisasi kode.

Portofolio produk Watsonx Code Assistant akan terus berkembang seiring waktu untuk mencakup bahasa pemrograman lain, mengatasi tantangan yang dihadapi oleh pengembang, dan meningkatkan waktu untuk mencapai nilai dalam upaya modernisasi.

Watsonx Code Assistant for Z sedang dikembangkan untuk membantu bisnis memanfaatkan kecerdasan buatan generatif dan alat otomatis guna mempercepat modernisasi aplikasi mainframe mereka, sambil tetap mempertahankan kinerja, keamanan, dan ketahanan IBM Z.

Dengan menggunakan watsonx Code Assistant for Z secara besar-besaran, pengembang dapat secara selektif dan bertahap mengubah layanan bisnis COBOL menjadi kode Java yang terstruktur dengan baik dan berkualitas tinggi. Dengan potensi miliaran baris kode COBOL sebagai kandidat untuk modernisasi yang ditargetkan seiring waktu, AI generatif dapat membantu menilai, memperbarui, memvalidasi, dan menguji kode yang sesuai dengan cepat. Hal ini memungkinkan modernisasi aplikasi besar secara lebih efisien dan memungkinkan pengembang untuk fokus pada tugas-tugas yang memiliki dampak lebih besar.

Langkah-langkah kunci dalam proses ini meliputi refactoring layanan bisnis dalam COBOL, mengubah kode COBOL menjadi kode Java yang dioptimalkan, dan memvalidasi hasilnya melalui pengujian otomatis.

Manfaat potensial bagi klien meliputi:

- Perkembangan kode yang lebih cepat dan peningkatan produktivitas pengembang sepanjang siklus modernisasi aplikasi
- Pengelolaan biaya, kompleksitas, dan risiko yang terkait dengan inisiatif modernisasi, termasuk terjemahan dan optimasi kode
- Akses yang lebih luas ke pool keterampilan IT yang lebih besar dan proses onboarding pengembang yang lebih cepat
- Peningkatan kualitas dan kode yang mudah dipelihara melalui penyesuaian model dan penerapan praktik terbaik

Kode Java yang dihasilkan dari watsonx Code Assistant for Z akan bersifat berorientasi objek dan dioptimalkan untuk interoperabilitas dengan sisa aplikasi

COBOL, termasuk CICS, IMS, DB2, dan runtime z/OS lainnya. Java on Z dirancang untuk memberikan optimasi kinerja dibandingkan dengan platform x86.

2 Generative AI for Software Development Workflows and its Considerations

2.1 Definisi CI/CD

Continuous Integration/Continuous Deployment adalah metode untuk secara berkala mengirimkan aplikasi dengan memanfaatkan otomatisasi.

2.2 Penggunaan AI dalam CI/CD

- Automated Testing and Quality Assurance
AI-driven testing tools mengotomatiskan proses pengujian untuk memastikan kualitas perangkat lunak dengan mengidentifikasi masalah. AI tools dapat digunakan untuk mengurangi waktu pengujian dan usaha manual.
- Code Analysis and Optimization
AI tools menggunakan algoritma machine learning untuk mendeteksi pola dan masalah dalam kode, serta merekomendasikan optimasi. AI tools dapat meningkatkan kualitas kode dan keamanan.
- Software Deployment
Predictive analytics yang didukung oleh AI menganalisis data historis untuk memprediksi risiko dalam proses deployment. AI models membantu dalam mengoptimalkan strategi deployment.
- Intelligent Release Orchestration
AI-driven release orchestration tools menjadwalkan deployment komponen perangkat lunak dengan memahami ketergantungan dan batasan sumber daya. AI tools dapat menyesuaikan sumber daya berdasarkan permintaan.
- Continuous Monitoring and Feedback
AI-based monitoring tools mendeteksi perilaku abnormal dan ancaman keamanan secara instan. Sintaks: AI analyzes user feedback untuk memberikan wawasan bagi pengembangan fitur di masa depan.

2.3 Beberapa alat CI/CD yang didukung AI

- Jenkins: Server otomatisasi open-source.
- IBM Watson Studio: Menawarkan otomatisasi DevOps untuk alur kerja ML.
- GitLab CI/CD: Mengintegrasikan dengan kontrol versi untuk manajemen CI/CD yang lebih baik.

2.4 Tren Masa Depan dalam CI/CD

- Operationalization AI: Solusi monitoring dan observability yang mengintegrasikan kemampuan AI untuk meningkatkan pemahaman tentang kinerja aplikasi.
- Delivery Health Insights: AI dan teknik ML memberikan wawasan lebih dalam tentang kualitas rilis aplikasi.
- Automated Verification: Teknik seperti natural language processing dan pattern recognition mempercepat analisis data dari berbagai alat CI/CD.

2.5 Penggunaan Gen AI dalam Keamanan Perangkat Lunak

- Deteksi Kerentanan: Generative AI dapat membantu mendeteksi kerentanan lebih awal dalam proses pengembangan dengan memindai kode untuk mengidentifikasi risiko keamanan.
- Pembuatan Kode Aman: Alat AI dapat menghasilkan potongan kode yang aman secara otomatis dan memberikan rekomendasi untuk perbaikan.

2.6 Alat dan Fungsionalitas AI dalam Keamanan

- Evaluasi Kode: Pengembang dapat mengunggah file kode lengkap untuk dievaluasi oleh alat AI, yang dapat menganalisis aliran data dan menemukan kerentanan.
- Model API dan Data: Alat AI dapat digunakan untuk mengevaluasi respons API atau model data untuk menemukan kesalahan atau bug.

2.7 Praktik Terbaik dalam Integrasi Gen AI

- Integrasi Berkelanjutan: Penting untuk terus mengintegrasikan praktik Generative AI ke dalam proyek pengembangan perangkat lunak, dimulai dari proyek kecil hingga yang lebih besar.
- Penggunaan Alat Seperti GitHub Copilot: Alat ini dapat membantu dalam pembuatan kode dan integrasi dengan repositori kontrol versi.

2.8 Tantangan dan Pertimbangan Etis

- Bukan Solusi Sempurna: Generative AI tidak dapat menyelesaikan semua masalah; alat ini lebih efektif dalam ruang masalah yang sempit.
- Pengawasan Manusia: Penting untuk meninjau hasil yang dihasilkan oleh AI secara berkala untuk menjaga kualitas dan mencegah ketergantungan berlebihan pada proses otomatis.

2.9 Peran AI dalam Keamanan Perangkat Lunak

AI berfungsi untuk meningkatkan keamanan perangkat lunak dengan mendeteksi dan mencegah ancaman secara otomatis. Contoh: Penyedia email menggunakan AI untuk menyaring email spam dan phishing, menjaga keamanan pengguna.

2.10 Aplikasi AI dalam Cybersecurity

- Automated Code Reviews: AI membantu menemukan potensi kerentanan dalam kode selama pengembangan.
- Static Application Security Testing (SAST): Memindai kode sumber untuk mengidentifikasi ancaman nyata dan mengurangi false positives.
- Dynamic Application Security Testing (DAST): Mensimulasikan serangan untuk menemukan kerentanan saat runtime.
- User and Entity Behavior Analytics (UEBA): Mendeteksi aktivitas mencurigakan selama proses pengembangan.

2.11 Kelemahan AI dalam Cybersecurity

- Resource Intensity: AI memerlukan sumber daya besar, termasuk kapasitas penyimpanan data dan kemampuan komputasi.
- Data Prerequisites: Diperlukan dataset yang beragam untuk melatih sistem AI, yang bisa sulit didapat.
- Cyber Intrusions: Penyerang dapat memanfaatkan AI untuk meningkatkan malware, menciptakan ancaman yang lebih canggih.

2.12 Integrasi AI dalam Pengembangan Perangkat Lunak

Kode yang dihasilkan oleh AI dapat mengandung kerentanan jika tidak divalidasi dengan baik, seperti SQL injection dan cross-site scripting (XSS).

2.13 Alat Keamanan Kode

- Qwiet AI Pre-Zero: Mengintegrasikan keamanan ke dalam CI/CD pipelines dan alat pengembangan, memberikan umpan balik cepat untuk mengidentifikasi kerentanan kode.
- Snyk Code: Menganalisis kode dengan cepat dan memberikan wawasan yang dapat ditindaklanjuti untuk membantu insinyur mengatasi masalah kode.
- Sophos Intercept X: Solusi perlindungan endpoint yang menggunakan teknologi pembelajaran mendalam untuk mengidentifikasi dan merespons ancaman.

- Symantec Endpoint Security: Alat yang didukung AI yang menggunakan pembelajaran mesin untuk mengidentifikasi kerentanan dan mengambil langkah pencegahan sebelum serangan terjadi.
- Splunk User Behavior Analytics: Fokus pada perilaku pengguna untuk mendeteksi potensi pelanggaran dengan menganalisis perbedaan dalam aktivitas pengguna.
- Vectra Threat Detection and Response: Menggunakan analitik perilaku untuk mendeteksi dan menghentikan ancaman di berbagai lingkungan.
- IBM QRadar Advisor with Watson: Mengotomatiskan operasi pusat keamanan dan menganalisis anomali untuk memberikan analisis penyebab tanpa intervensi manusia.
- Microsoft Security Copilot: Membantu tim keamanan dalam menyelidiki dan merespons insiden keamanan.
- BurpGPT: Plugin untuk Burp Suite yang menggunakan GPT untuk menganalisis permintaan dan respons HTTP.
- EscalateGPT: Mengidentifikasi kerentanan eskalasi hak istimewa menggunakan GPT dalam kebijakan manajemen akses identitas yang salah konfigurasi.

2.14 Istilah Teknis Security

- SQL Injection: Serangan yang memungkinkan penyerang untuk mengakses database dengan menyisipkan kode SQL berbahaya.
- Cross-Site Scripting (XSS): Kerentanan yang memungkinkan penyerang menyisipkan skrip berbahaya ke dalam halaman web yang dilihat oleh pengguna lain.

2.15 Penerapan AI dalam Pengujian Perangkat Lunak

- AI Techniques: Teknik AI seperti machine learning, natural language processing (NLP), dan intelligent automation digunakan untuk meningkatkan pengujian perangkat lunak.
- Test Data Generation: AI dapat menghasilkan data uji yang beragam dan representatif dari skenario dunia nyata, mengurangi waktu dan usaha yang diperlukan untuk menghasilkan data uji.

2.16 Natural Language Processing (NLP) dalam Pengujian

- Automated Test Case Generation: NLP dapat secara otomatis mengekstrak informasi dari dokumen persyaratan dan menghasilkan test cases.

- **Keyword Extraction:** Sistem berbasis NLP dapat mengidentifikasi kata kunci dan hubungan antar komponen dalam dokumen persyaratan.

2.17 Contoh Kasus Penggunaan AI dalam Testing

- **Test Data Synthesis:** Menggunakan algoritma AI untuk menganalisis data yang ada dan menghasilkan data baru yang mirip dengan pola yang teramati.
- **Automated Test Input Generation:** Algoritma machine learning dapat menghasilkan input uji secara otomatis berdasarkan perilaku perangkat lunak yang dipahami.

2.18 Praktik Terbaik dalam Menggunakan AI untuk Pengujian

- **Controlled Environment:** Mulailah dengan eksperimen teknik AI dalam lingkungan yang terkontrol sebelum memperluas penggunaannya.
- **Human Oversight:** Meskipun AI dapat mengotomatiskan banyak aspek pengujian, pengawasan manusia tetap penting untuk memvalidasi hasil dan membuat keputusan yang tepat.

2.19 AI dalam Software Code Review

AI membantu meningkatkan akurasi dan efisiensi dalam code review dengan mengotomatiskan tugas dan analisis. Teknik:

- **Static Analysis:** Menganalisis kode sumber tanpa mengeksekusinya untuk menemukan masalah potensial seperti pelanggaran gaya kode dan keborcoran memori.
- **Bug Detection:** Menggunakan algoritma machine learning untuk memprediksi bug berdasarkan perbaikan bug sebelumnya.

2.20 AI dalam Software Debugging

AI membantu mengidentifikasi dan memberikan saran untuk perbaikan bug yang sulit ditemukan dengan metode tradisional. Teknik:

- **Automated Log Analysis:** Menganalisis log untuk menemukan pola yang menunjukkan adanya bug.
- **Predictive Debugging:** Memanfaatkan data historis untuk memprediksi penyebab bug berdasarkan kejadian sebelumnya.

2.21 AI dalam Software Documentation

AI menyederhanakan proses dokumentasi dengan menghasilkan konten yang jelas dan akurat.

- Natural Language Processing (NLP): Menganalisis komentar kode dan pesan komit untuk menghasilkan dokumentasi secara otomatis.
- API Documentation Generation: Menghasilkan dokumentasi API dengan menganalisis struktur kode.

2.22 AI dalam Pendidikan dan Pelatihan

AI menciptakan pengalaman belajar yang dipersonalisasi dan adaptif.

- Personalized Learning: Menyesuaikan materi pelatihan berdasarkan kebutuhan dan preferensi individu.
- Adaptive Training: Mengubah tingkat kesulitan latihan berdasarkan kinerja peserta.

2.23 Manfaat dan tantangan penggunaan Kecerdasan Buatan Generatif dalam pengembangan perangkat lunak

Penggunaan kecerdasan buatan generatif (generative AI) dalam pengembangan perangkat lunak menawarkan beberapa manfaat. Pertama, kecerdasan buatan generatif menyediakan alat yang kuat untuk memperluas kumpulan data yang ada dan menghasilkan data sintetis, sehingga mengurangi kebutuhan akan jumlah besar data berlabel. Alat ini dapat secara signifikan mempercepat siklus pengembangan dan menurunkan biaya.

Selain itu, kecerdasan buatan generatif memungkinkan kreativitas dan eksplorasi dalam pengembangan perangkat lunak. Alat ini membantu menghasilkan solusi atau konten baru yang mungkin tidak mungkin dicapai dengan pendekatan tradisional. Hal ini dapat menghasilkan aplikasi inovatif dan pengalaman pengguna yang unik.

Namun, terdapat juga tantangan yang terkait dengan penggunaan AI generatif. Salah satu tantangan utama adalah potensi bias dan ketidakakuratan dalam output yang dihasilkan. Model AI generatif belajar dari data yang ada, dan jika data pelatihan bias atau tidak lengkap, konten yang dihasilkan mungkin menunjukkan bias atau ketidakakuratan serupa.

Tantangan lain adalah pertimbangan etis seputar AI generatif. Misalnya, menghasilkan gambar atau video palsu yang realistis dapat menimbulkan kekhawatiran tentang privasi, disinformasi, dan potensi penyalahgunaan. Penting untuk mempertimbangkan implikasi etis ini saat menggunakan AI generatif.

2.24 Pertimbangan hak kekayaan intelektual

Saat menggunakan kecerdasan buatan generatif (generative AI) dalam pengembangan perangkat lunak, penting untuk memahami implikasi hak kekayaan intelektual (HKI) dan mengambil langkah-langkah yang tepat untuk melindungi hak-hak Anda. Kecerdasan buatan generatif dapat menghasilkan konten yang dilindungi oleh undang-undang hak kekayaan intelektual, seperti hak cipta atau paten. Misalnya, jika sistem kecerdasan buatan generatif menghasilkan kode unik, kode tersebut mungkin dilindungi oleh hak cipta.

- Memahami kerangka hukum yang mengatur hak kekayaan intelektual dan bagaimana hal itu berlaku untuk konten yang dihasilkan oleh kecerdasan buatan generatif (generative AI) sangat penting. Kesadaran ini mencakup pemahaman tentang hak kekayaan intelektual yang sudah ada dan potensi masalah pelanggaran. Hak cipta melindungi karya asli yang diciptakan dan diabadikan dalam media ekspresi yang tangible. Dalam konteks kecerdasan buatan generatif, hak cipta dapat berlaku untuk konten yang dihasilkan jika memenuhi syarat perlindungan. Namun, hak cipta umumnya dimiliki oleh pencipta manusia yang melatih atau memprogram sistem kecerdasan buatan generatif tersebut, bukan sistemnya sendiri.
- Paten melindungi penemuan atau proses yang baru, berguna, dan tidak jelas. Meskipun algoritma AI generatif sendiri kemungkinan besar tidak dapat dipatenkan, aplikasi spesifik atau teknik baru yang dikembangkan menggunakan algoritma tersebut mungkin memenuhi syarat untuk perlindungan paten.
- Lisensi juga memainkan peran krusial dalam konteks AI generatif. Pengembang harus mempertimbangkan perjanjian lisensi saat menggunakan model yang telah dilatih sebelumnya atau kumpulan data untuk memastikan kepatuhan terhadap batasan atau persyaratan yang ditetapkan oleh syarat lisensi.

2.24.1 Strategi untuk melindungi hak kekayaan intelektual saat menggunakan kecerdasan buatan generatif

- Tentukan kepemilikan dengan jelas: Tetapkan hak kepemilikan atas konten yang dihasilkan dan jelaskan siapa yang memegang hak cipta atau hak paten.
- Simpan catatan: Simpan catatan detail tentang proses pelatihan, termasuk dataset yang digunakan dan langkah-langkah yang diambil selama pengembangan model.
- Pantau pelanggaran: Pantau secara rutin potensi pelanggaran terhadap konten yang dihasilkan dan ambil tindakan yang sesuai jika diperlukan.

- Gunakan watermark atau metadata: Pertimbangkan untuk menambahkan watermark atau metadata pada output yang dihasilkan untuk menunjukkan kepemilikan dan memastikan jejak audit.
- Minta nasihat hukum: Konsultasikan dengan ahli hukum yang berpengalaman dalam hukum kekayaan intelektual untuk memastikan kepatuhan dan melindungi hak Anda secara efektif.

2.25 Pertimbangan keamanan

Sistem kecerdasan buatan generatif (Generative AI) bergantung pada jumlah data yang besar untuk pelatihan. Data ini dapat mencakup informasi sensitif yang dapat menimbulkan risiko keamanan jika tidak ditangani dengan benar. Misalnya, model kecerdasan buatan generatif yang dilatih menggunakan data pengguna pribadi tanpa perlindungan yang memadai dapat secara tidak sengaja mengungkapkan informasi sensitif. Selain itu, model kecerdasan buatan generatif itu sendiri dapat rentan terhadap serangan. Contoh-contoh adversarial dapat dimanfaatkan untuk memanipulasi output sistem kecerdasan buatan generatif, yang berpotensi menyebabkan pelanggaran keamanan atau penyebaran informasi yang menyesatkan.

2.25.1 Mengurangi risiko kebocoran data dan akses tidak sah ke sistem kecerdasan buatan generatif

Untuk mengurangi risiko kebocoran data dan akses tidak sah ke sistem kecerdasan buatan generatif, gunakan langkah-langkah berikut:

- Enkripsi: Gunakan teknik enkripsi untuk melindungi data sensitif selama penyimpanan dan transmisi.
- Infrastruktur aman: Gunakan infrastruktur aman dengan kontrol akses yang tepat dan kemampuan pemantauan.
- Autentikasi pengguna: Terapkan mekanisme autentikasi yang kuat untuk memastikan hanya individu yang berwenang yang dapat mengakses sistem kecerdasan buatan generatif.
- Audit rutin: Lakukan audit rutin terhadap log sistem dan catatan akses untuk mengidentifikasi aktivitas mencurigakan atau potensi pelanggaran.
- Rencana tanggap insiden: Buat rencana tanggap insiden yang menjelaskan langkah-langkah yang harus diambil jika terjadi pelanggaran keamanan atau akses tidak sah.

2.26 Pertimbangan kepatuhan

Aplikasi kecerdasan buatan generatif sering kali menangani informasi sensitif, sehingga kepatuhan terhadap persyaratan hukum dan regulasi menjadi sangat

penting. Beberapa undang-undang yang berlaku meliputi Peraturan Perlindungan Data Umum (GDPR) atau peraturan perlindungan data lainnya yang spesifik untuk yurisdiksi tertentu. Selain itu, pertimbangkan aspek etika terkait keadilan, akuntabilitas, dan transparansi saat mengembangkan sistem kecerdasan buatan generatif.

Untuk memastikan kepatuhan terhadap standar perlindungan data, privasi, dan etika saat menggunakan kecerdasan buatan generatif, pertimbangkan untuk menerapkan praktik-praktik berikut:

- Persetujuan dan transparansi: Peroleh persetujuan yang terinformasi dari individu yang datanya digunakan dalam pelatihan atau pembuatan konten dengan sistem kecerdasan buatan generatif. Jelaskan bagaimana Anda akan menggunakan data mereka.
- Privasi sejak awal: Terapkan langkah-langkah yang meningkatkan privasi dalam desain sistem kecerdasan buatan generatif sejak awal.
- Anonimisasi dan pseudonimisasi: Terapkan teknik seperti anonimisasi atau pseudonimisasi untuk melindungi identitas individu dalam output yang dihasilkan.
- Keadilan dan mitigasi bias: Secara rutin menilai dan mengatasi bias yang timbul dari data pelatihan atau algoritma untuk memastikan keadilan dalam output yang dihasilkan.
- Keterjelasan: Usahakan keterjelasan dan keterbacaan dalam sistem AI generatif dengan mengadopsi teknik yang memberikan wawasan tentang proses pengambilan keputusan.

2.27 Mengintegrasikan transparansi dan akuntabilitas ke dalam sistem kecerdasan buatan generatif

Mengintegrasikan transparansi dan akuntabilitas ke dalam sistem kecerdasan buatan generatif melibatkan:

- Dokumentasi: Jaga dokumentasi rinci tentang proses pengembangan, termasuk sumber data pelatihan, arsitektur model, hiperparameter, dan langkah-langkah pra-pemrosesan.
- Penjelasan model: Jelajahi teknik yang memberikan penjelasan tentang keputusan yang diambil oleh sistem kecerdasan buatan generatif.
- Audit: Lakukan audit secara berkala terhadap sistem AI generatif untuk memastikan kepatuhan terhadap standar etika dan mengidentifikasi area yang perlu ditingkatkan.
- Peninjauan eksternal: Mintalah peninjauan atau validasi eksternal dari ahli atau badan regulasi untuk memastikan transparansi dan akuntabilitas.

2.28 Menangani ketidakakuratan dan bias

Model kecerdasan buatan generatif memiliki kemampuan yang luar biasa, tetapi tidak kebal terhadap ketidakakuratan atau bias. Penting untuk menangani masalah-masalah ini dengan tepat.

Model AI generatif belajar dari data pelatihan, artinya mereka dapat mewarisi bias apa pun yang terdapat dalam data tersebut. Demikian pula, ketidakakuratan dapat muncul jika data pelatihan tidak mewakili distribusi output yang diinginkan dengan baik. Pengembang harus memahami potensi masalah ini saat bekerja dengan model AI generatif untuk menghindari konsekuensi yang tidak diinginkan atau output yang tidak adil. Untuk mengevaluasi dan menangani ketidakakuratan dan bias dalam sistem AI generatif, sertakan:

- Data pelatihan yang beragam: Pastikan kumpulan data pelatihan mewakili distribusi data keluaran yang diinginkan.
- Deteksi dan mitigasi bias: Terapkan teknik untuk mendeteksi bias dalam keluaran yang dihasilkan dan kembangkan strategi mitigasi.
- Metrik evaluasi: Tentukan metrik evaluasi yang sesuai untuk menilai kualitas keluaran yang dihasilkan secara objektif.
- Peningkatan berulang: Pantau dan sempurnakan model AI generatif secara terus-menerus berdasarkan umpan balik pengguna atau hasil evaluasi.
- Siklus umpan balik pengguna: Bangun mekanisme umpan balik di mana pengguna dapat melaporkan ketidakakuratan atau bias yang mereka temui saat berinteraksi dengan output yang dihasilkan.

2.29 Pertimbangan Etis dalam Penggunaan Kecerdasan Buatan Generatif pada Pengembangan Perangkat Lunak

Kecerdasan Buatan Generatif (Generative AI), dengan kemampuannya untuk menciptakan konten mulai dari teks hingga gambar, telah merevolusi bidang pengembangan perangkat lunak. Seiring dengan para pengembang yang memanfaatkan kekuatan model AI ini untuk mengotomatisasi berbagai tugas dan meningkatkan pengalaman pengguna, menjadi penting untuk mempertimbangkan aspek etika yang terkait dengan penggunaannya:

- Bias dan diskriminasi

Model kecerdasan buatan generatif dilatih menggunakan sejumlah besar data, yang secara tidak sengaja dapat mengandung bias yang terdapat dalam data pelatihan. Hal ini dapat menyebabkan pembangkitan konten yang bias atau diskriminatif. Misalnya, model bahasa yang dilatih menggunakan dataset yang mengandung bahasa bias dapat menghasilkan

output yang memperkuat stereotip atau mendiskriminasi kelompok tertentu. Untuk mengatasi masalah ini, pengembang harus dengan cermat mengkurasi dan memproses data pelatihan untuk meminimalkan bias dan memastikan keadilan.

Salah satu contoh yang menonjol dalam menangani bias adalah upaya OpenAI dengan ChatGPT. Selama fase pratinjau penelitian, OpenAI menemukan bahwa model tersebut menunjukkan perilaku bias saat diberi masukan tertentu. Untuk mengatasi hal ini, digunakan API Moderasi untuk memperingatkan atau memblokir jenis konten yang tidak aman tertentu. Pendekatan proaktif ini menunjukkan komitmen dalam menangani bias dan diskriminasi.

- Hak kekayaan intelektual

Model kecerdasan buatan generatif memiliki potensi untuk menghasilkan konten yang dapat melanggar hak kekayaan intelektual. Misalnya, model generasi gambar dapat secara tidak sengaja menghasilkan materi yang dilindungi hak cipta. Pengembang perlu menyadari risiko semacam ini dan memastikan bahwa konten yang dihasilkan sesuai dengan undang-undang hak kekayaan intelektual.

Sebagai contoh, gunakan Looka, alat berbasis AI untuk pembuatan logo. Meskipun alat ini mempermudah proses desain logo, pengembang dan pengguna harus berhati-hati dalam memasukkan elemen berhak cipta ke dalam logo mereka. Dengan memahami undang-undang hak cipta dan menghormati hak kekayaan intelektual, pengembang dapat menghindari masalah hukum.

- Privasi dan perlindungan data

Model kecerdasan buatan generatif memerlukan data pelatihan yang luas, yang sering kali mencakup informasi sensitif atau pribadi. Pengembang harus menangani data ini dengan hati-hati untuk melindungi privasi pengguna dan mematuhi peraturan perlindungan data seperti GDPR atau CCPA. Penting untuk mendapatkan persetujuan yang terinformasi dari pengguna terkait penggunaan data dan memastikan praktik penyimpanan dan pemrosesan data yang aman.

Contoh model AI generatif yang memperhatikan privasi adalah Deep Nostalgia. Alat ini memungkinkan pengguna untuk menganimasi foto keluarga historis. Namun, sangat penting untuk mendapatkan persetujuan yang sesuai dari individu yang tampil dalam foto-foto tersebut sebelum menggunakan penampilan mereka untuk animasi. Menghormati hak privasi memastikan bahwa AI generatif digunakan secara etis dan bertanggung jawab.

- Disinformasi dan manipulasi

Model kecerdasan buatan generatif (Generative AI) memiliki potensi untuk menciptakan konten yang realistis dan mudah disalahartikan sebagai

konten asli yang dibuat oleh manusia. Hal ini menimbulkan kekhawatiran terkait disinformasi dan manipulasi. Pengembang harus berhati-hati dalam penggunaan Generative AI untuk menghindari penyebaran informasi palsu atau manipulasi pengguna.

Contoh penanganan informasi yang menyesatkan adalah Legal Robot, alat AI yang menyederhanakan bahasa hukum. Dengan menerjemahkan istilah hukum yang kompleks ke dalam bahasa yang mudah dipahami, alat ini bertujuan untuk meningkatkan pemahaman hukum bagi profesional maupun awam. Dengan memastikan akurasi dan transparansi dalam konten yang dihasilkan, pengembang dapat melawan informasi yang menyesatkan.

- Akuntabilitas dan transparansi

Seiring dengan semakin luasnya penggunaan kecerdasan buatan generatif (AI) dalam pengembangan perangkat lunak, sangat penting untuk menetapkan kerangka kerja akuntabilitas dan transparansi. Pengembang harus secara jelas mengkomunikasikan kapan konten yang dihasilkan oleh AI digunakan dan memberikan informasi kepada pengguna tentang cara konten tersebut dihasilkan. Hal ini memberdayakan pengguna untuk membuat keputusan yang terinformasi dan mengurangi risiko konsekuensi yang tidak diinginkan.

Contoh yang sangat baik tentang akuntabilitas adalah Murf, mesin teks-ke-suara yang memungkinkan pengguna membuat rekaman suara sintesis. Dengan secara jelas menyatakan bahwa output dihasilkan oleh AI dan menyediakan opsi untuk memilih suara dan dialek yang berbeda, Murf mempromosikan transparansi dan memastikan pengguna memahami sifat konten yang mereka buat.

3 Final Project and Final Exam